RoboGuardian: Fault-Aware Execution and Memory Maintenance for Embodied Agent Harness Frameworks

Zijie Gao*
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
zijiegao@smail.nju.edu.cn

Chenxu Li*
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
chenxuli@smail.nju.edu.cn

Zixuan Chen†
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
chenzx@nju.edu.cn

Jiapeng Xu
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
522025330117@smail.nju.edu.cn

Wenquan Zhang
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
652026330034@smail.nju.edu.cn

Yizhou Zhu
University of Pennsylvania
Philadelphia, USA
jyz9029@engineering.upenn.edu

Rongjin Zheng
Central South University
Changsha, China
rongjinzheng@csu.edu.cn

Xinke Wang
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
522026330171@smail.nju.edu.cn

Jieqi Shi†
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
isjieqi@nju.edu.cn

Jing Huo†
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
huojing@nju.edu.cn

Yang Gao
State Key Laboratory of Novel Software Technology, Nanjing University
Nanjing, China
gaoy@nju.edu.cn

Abstract

Embodied-agent harnesses connect high-level planners to low-level execution policies and use execution results to update task state and memory. However, current harness mechanisms often fail to verify physical state changes reliably. This limitation applies to both low-level execution feedback and VLM-based evaluations. Unverified outcomes may therefore enter memory and mislead subsequent actions. In this paper, we present **RoboGuardian**, a fault-aware execution and memory-maintenance framework for embodied-agent harnesses. It organizes execution exceptions into three response-oriented categories: outdated plan steps, uncertain outcomes, and execution failures. This categorization enables robust decision-making from multi-modal evidence. RoboGuardian checks plan validity and entity identity before execution and combines semantic, proprioceptive, and effect-level evidence to produce a three-way physical-effect judgment where only physically confirmed actions update its embodied memory and progress. It also maintains a bounded trusted context and re-observes before retrying unresolved actions, preventing blind retry and replay. We evaluate RoboGuardian across diverse simulation environments, including robotic arm manipulation on LIBERO-10, dexterous grasping on Clutter-Bench, and continuous vision-language navigation in Habitat. Furthermore, we deploy the system in the real world on both dual-arm and dexterous-hand manipulation platforms. Experimental results demonstrate that RoboGuardian effectively enhances execution reliability across diverse tasks and hardware embodiments. The framework maintains a lightweight and effective memory mechanism while introducing minimal computational overhead during execution. Videos and open-source code can be found on our project website: https://tsuki-gor.github.io/RoboGuardian/.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; Robotic planning; • **Computer systems organization** → *Robotic autonomy*.

Keywords

embodied agents, agent harness, execution fault detection, failure recovery, robot memory

*Both authors contributed equally to this research.
†Corresponding authors.

DAI '26, Hong Kong, China

ACM Reference Format:
Zijie Gao, Chenxu Li, Zixuan Chen, Jiapeng Xu, Wenquan Zhang, Yizhou Zhu, Rongjin Zheng, Xinke Wang, Jieqi Shi, Jing Huo, and Yang Gao. 2026. RoboGuardian: Fault-Aware Execution and Memory Maintenance for Embodied Agent Harness Frameworks. In *The Eighth International Conference on Distributed Artificial Intelligence (DAI '26), November 29–December 2, 2026, Hong Kong, China.* ACM, New York, NY, USA, 11 pages.

1 Introduction

Advances in learned visuomotor policies and vision-language-action models have enabled general-purpose language-conditioned manipulation [2, 5, 7, 10, 23, 46, 49]. In parallel, language-model planners compose and invoke motor skills [1, 4, 6, 8, 19, 28]. Embodied systems rely on harness architectures to integrate language-model planning with perception, actuation, and heterogeneous hardware through a unified interface [3, 31, 41]. These architectures commonly follow a "Plan and Execute" paradigm: an agent interprets an instruction, a planner translates it into motor sequences, and hardware executes the corresponding skills. Yet existing harnesses often lack explicit mechanisms linking post-execution verification to trusted state and memory updates.

Prior work seeks to augment execution with vision-language success and failure detection [13, 14], experience summaries [15, 33], visual monitors [26, 48], or explicit memory managers [20, 35, 42, 45, 47]. However, these capabilities are typically handled separately, and many systems lack an explicit shared rule governing action dispatch, trusted-state updates, and retries. Coordinating these decisions requires addressing three fundamental challenges. First, software signals often conflict with physical outcomes, obscuring the true environmental state. Second, indiscriminately recording unverified results in memory propagates errors that undermine long-horizon tasks. Third, generic retry strategies are inadequate and potentially dangerous, as distinct exceptions, such as outdated plans or execution failures, require different recovery responses.

To address these challenges, we propose **RoboGuardian**, a framework for fault-aware execution and memory maintenance in embodied-agent harnesses. As shown in Fig. 1, RoboGuardian extends the basic harness architecture into four connected stages. It retains the standard *Plan and Execute* workflow for task understanding and action proposal and introduces three post-execution capabilities corresponding to the challenges above: (1) Execution result recognition and classification (***Recognize***): treating software returns as untrusted and combining available multimodal evidence to assess whether the intended physical effect occurred, yielding *yes*, *no*, or *uncertain*; (2) Lightweight memory updating and maintenance (***Remember***): using a commit gate to ensure that only verifier-accepted outcomes update trusted memory and task progress, preventing error propagation; and (3) Failure-category-based recovery (***Recovery***): mapping each recognized category to an appropriate response, such as replanning, alternative skill selection, or re-observation, rather than blindly retrying the action. We treat the planner and skill library as given, focusing on the authority boundary between a skill return and downstream decisions that may treat it as a trusted physical fact.

We enforce this boundary through an *Effect Receipt* mechanism coupled with a gated memory system. Because many physical skills are non-idempotent, blind retries under uncertain outcomes can duplicate actions and pose greater risks than pausing. The Receipt mechanism requires re-observation before retrying and admits only Committed receipts to trusted memory. Rejected and Unknown attempts are isolated in an Attempt Log for recovery diagnosis, preventing them from being mistaken for task progress. By organizing verified facts, verified progress, and relation memory into a bounded context, this design limits the long-term impact of execution errors and controls context costs during long-horizon tasks.

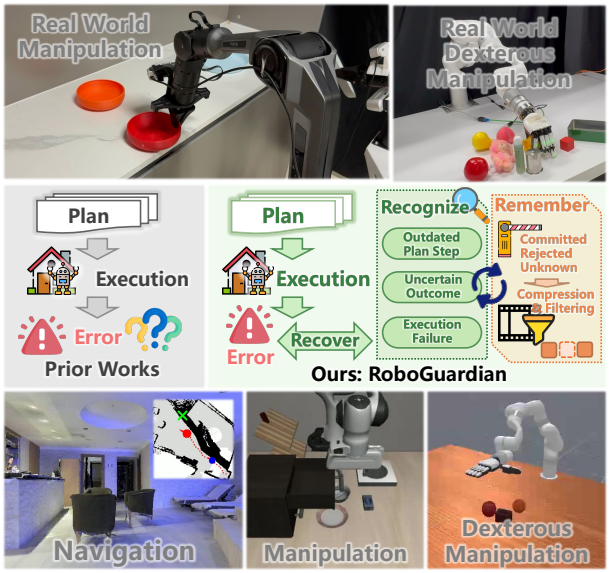


Figure 1: Prior embodied-agent harnesses may write unverified runtime results into task state and memory, allowing errors to propagate. RoboGuardian checks actions before dispatch, verifies physical effects from multimodal evidence, gates memory updates, and re-observes the environment before retrying, enabling robust execution across diverse robot embodiments.

Final task success alone is insufficient to evaluate such a harness. Consider a rollout in which a skill response is lost, the system repeats a non-idempotent action, and the final environment happens to satisfy the task condition. A conventional success metric counts this rollout as successful but overlooks the duplicate action or incorrect memory. We therefore report verified success rate alongside task success rate, as well as unverified memory writes, duplicate physical actions, recovery coverage, recovery success, and context cost. These metrics jointly assess task outcomes, execution reliability, and harness operating costs.

Our contributions are as follows:

- We analyze how embodied-agent harnesses misjudge skill outcomes and derive a taxonomy of outdated plan steps, uncertain outcomes, and execution failures. Defined by required responses rather than physical root causes, these categories are decidable from evidence available at decision time.

- We introduce RoboGuardian, which integrates action admission, multimodal physical-effect verification, effect-receipt-gated memory, and typed recovery into a unified pipeline. It prevents physical retries under insufficient evidence and controls memory size through a bounded context.
- We evaluate RoboGuardian in simulation on LIBERO-10 [30], on Clutter-Bench [9] for dexterous grasping, and in Habitat [11, 25, 40] for continuous vision-language navigation. We further demonstrate RoboGuardian on two real-world platforms, a dexterous hand and a gripper-based manipulator, providing qualitative evidence of perception-derived verification, recovery, and memory maintenance.

2 Related Work

Table 1 compares action admission, outcome recognition, recovery, and memory mechanisms across three areas of related work. Embodied-agent harnesses (Sec. 2.1) focus on skill selection and execution coordination, with less emphasis on when outcomes can be accepted as verified physical facts. Execution monitors (Sec. 2.2) detect or explain abnormal execution but do not always specify how verification governs trusted memory updates and subsequent execution. Agent memory systems (Sec. 2.3) preserve and retrieve extended context but generally assume stored outcomes are reliable. RoboGuardian links physical-effect verification to rules governing trusted memory updates and execution recovery.

2.1 Embodied Agent Harness Frameworks

Recent embodied systems often use language models to select and invoke parameterized robot skills. These models can generate actionable plans without task-specific training [17]. SayCan [1] grounds such plans with learned affordance scores. Code as Policies [28] generates executable programs over perception and control primitives. Other methods incorporate feasibility checks [29], use 3D scene graphs for large environments [37], or integrate perception into the planning model [12].

Other frameworks support long-horizon execution across heterogeneous robot embodiments. GUAVA [31] examines the action, observation, and workflow design of embodied tool use. RoboOS [41] coordinates heterogeneous robots and skills through a hierarchical architecture. RoboClaw, ROSClaw, and ABot-Claw explore long-horizon orchestration, ROS 2 execution, and cross-embodiment collaboration [3, 21, 27]. DeCo [8] decomposes demonstrations into atomic tasks and composes reusable skills using a VLM for zero-shot generalization to long-horizon 3D manipulation. MoMaStage [26] constrains planning and closed-loop execution with a skill-state graph. Underlying skill implementations range from imitation-learning and diffusion-based policies [10, 46] to vision-language-action models [2, 23, 39, 49]. Complementary work explores robotic simulation and trajectory generation, with V-Dreamer [16] using video generation priors to automate both. ClawKeeper [32] addresses software and system security for OpenClaw agents, a concern distinct from physical-effect verification.

These harnesses improve how skills are selected, constrained, and coordinated but do not centrally define what may be concluded after a skill returns. A single runtime result can conceal an outdated plan step, a lost response, or a missing physical effect. Forwarding it directly can trigger duplicate actions or record false prerequisites for later skills.

Table 1: Functional comparison with representative methods grouped by the three areas reviewed in Secs. 2.1–2.3. AG: action-admission gating; MA: multimodal assessment; EA: error-category attribution; TR: typed recovery feedback; RO: re-observation before retry; VM: verified-only memory; BM: bounded memory. ✓ denotes explicit support, ▲ partial support, and ✗ no central mechanism.

	Plan & Execute	*Recognize*		*Recovery*		*Remember*	
Method	AG	MA	EA	TR	RO	VM	BM
Sec. 2.1: Harnesses and skill invocation							
SayCan [1]	▲	✗	✗	✗	✗	✗	✗
Code as Policies [28]	✗	✗	✗	✗	▲	✗	✗
SayPlan [37]	▲	✗	✗	▲	✗	✗	▲
GUAVA [31]	▲	✓	✗	▲	✗	✗	✗
RoboOS [41]	▲	▲	✗	▲	✗	✗	✗
ROSClaw [3]	✓	▲	✗	▲	✗	✗	▲
DeCo [8]	▲	▲	✗	✗	✗	✗	✗
MoMaStage [26]	✓	▲	✗	▲	✗	✗	✗
Sec. 2.2: Execution monitoring							
Inner Monologue [19]	✗	▲	✗	✗	▲	✗	✗
DoReMi [15]	▲	▲	✗	▲	▲	✗	✗
REFLECT [33]	✗	✓	▲	▲	✗	✗	✗
AHA [14]	✗	▲	✓	✗	✗	✗	✗
Code-as-Monitor [48]	✓	✓	✗	▲	✗	✗	✗
Sec. 2.3: Agent memory							
Reflexion [38]	✗	✗	✗	▲	✗	✗	▲
Voyager [42]	✗	✗	✗	▲	✗	▲	▲
MemGPT [35]	✗	✗	✗	✗	✗	✗	✓
VLA memory [20]	✗	▲	✗	✗	✗	▲	▲
RoboGuardian (ours)	✓	✓	✓	✓	✓	✓	✓

2.2 Execution Failure Detection and Recovery

Another line of work adds explicit execution monitoring through different implicit verifiers. Inner Monologue [19] uses textual success detection and scene descriptions to provide planner feedback. Grounded Decoding [18] constrains generation with grounded model likelihoods. DoReMi [15] detects plan-execution misalignment and triggers recovery. Code-as-Monitor [48] compiles spatiotemporal constraints into visual programs for proactive and reactive monitoring. REFLECT [33] summarizes multimodal experience and uses a language model to explain failures and guide corrections. AHA [14] trains a vision-language model to detect and reason about manipulation failures. Other work studies VLM-based success detection [13] and explicit state tracking [45].

These methods demonstrate the value of execution feedback, but typically output success decisions, constraint violations, or language explanations. They rarely assign exceptions to categories with distinct authority consequences. For a grasp, an outdated action should be rejected before dispatch; a lost response should trigger re-observation before retry; and independent evidence of no grasp effect should lead to rejection at commit. Failure reports alone do not specify whether the system should replan, gather additional observations, or retry the action. Without a gate on task-state and memory updates, an incorrect success judgment can become a trusted prerequisite that later steps repeatedly retrieve.

2.3 Memory in Embodied-Agent Harnesses

Agent memory research mainly studies what to store, how to update it, and how to retrieve it. Representative methods record interleaved reasoning and action trajectories [44], retain verbal self-reflection across attempts [38], or summarize and retrieve observation streams [36]. Other approaches use tiered storage to overcome context-window limitations [35], introduce forgetting mechanisms [47], or accumulate reusable skills through continual exploration [42]. Long-term memory also supports open-world execution for vision-language-action agents [20]. These methods improve memory content, management, and retrieval but generally assume that recorded observations and action outcomes are reliable. This assumption is fragile in embodied execution, where software results may not reflect actual physical effects. Better retrieval cannot turn an unverified claim about the physical state into a reliable fact; it can only increase that claim's influence on subsequent decisions.

3 Method

We introduce RoboGuardian, a framework for fault-aware execution and memory maintenance in embodied-agent harnesses. It classifies execution errors by their required responses, performs lightweight memory updates, and applies tailored recovery strategies. Taking the planner and skill library as given, RoboGuardian checks proposed actions before dispatch, assesses physical effects after execution, and commits only verified effects. Software-level success alone is not treated as a physical fact, and unresolved outcomes require fresh evidence before any retry. Fig. 2 presents the four-stage pipeline.

3.1 Problem Formulation

A language-guided embodied agent operates over a set of robot embodiments $\mathcal{E}$. Each embodiment $b \in \mathcal{E}$ has a skill library $\mathcal{K}_b$, an action space, and an embodiment state. Given a language instruction ℓ and trusted context $\mathcal{B}_t$, the planner selects an embodiment and proposes an action:

$$a_t = \pi_{\text{plan}}(\ell, \mathcal{B}_t) = (b_t, k_t, \theta_t), \tag{1}$$

where $b_t \in \mathcal{E}$ is the selected embodiment, $k_t \in \mathcal{K}_{b_t}$ is one of its skills, and θ_t specifies parameters such as the target object and pose. The robot runtime executes a_t and returns a software status r_t. The system also receives pre- and post-execution observations, denoted by o_{t-1} and o_t, respectively, with

$$o_t = (\mathcal{I}_t, q_t^{b_t}), \tag{2}$$

where $\mathcal{I}_t$ is a visual observation and $q_t^{b_t}$ is the current embodiment state. The runtime status r_t reports an execution outcome but does not prove that the intended physical effect occurred. A successful gripper command does not guarantee a stable grasp, and a terminated navigation command does not guarantee that the base lies within the goal tolerance. Thus, r_t alone cannot authorize updates to task progress or trusted memory. We verify the intended effect using the action, software status, and observations:

$$z_t = \Phi(a_t, r_t, o_{t-1}, o_t) \in \{\text{yes}, \text{no}, \text{uncertain}\}. \tag{3}$$

The judgment yes indicates that the intended effect occurred, no that it was not achieved, and uncertain that evidence is insufficient. Only verified effects may update trusted state. Uncertain outcomes require re-observation before any physical retry.

3.2 Plan and Execute with Admission-Gated Action Dispatch

The admission module checks plan validity, skill applicability, and entity identity to make a pre-dispatch decision d_t:

$$d_t = \mathcal{A}(a_t, \mathcal{B}_t, o_{t-1}) \in \{\text{dispatch}, \text{reject}\}. \tag{4}$$

A proposal is classified as an outdated plan step when its preconditions, skill applicability, or entity identity no longer match the current state. It is rejected before dispatch, with typed feedback for replanning or policy re-inference from the latest observation. Identity checks cover the target object, selected embodiment, and skill parameters, preventing stale plans from acting on new scene instances or being dispatched to incompatible embodiments. An action enters the robot runtime only when $d_t = \text{dispatch}$. The runtime may report completion, timeout, or a low-level error. RoboGuardian retains these signals as evidence without treating them as physical facts. *Recognize* evaluates every dispatched action. Across the pipeline, exceptions are categorized by their required corrective responses: outdated plan steps are rejected before dispatch; uncertain outcomes ($z_t = \text{uncertain}$) trigger re-observation; and execution failures, assigned only when $z_t = \text{no}$, block positive commitment.

3.3 Recognize with Multimodal Physical-Effect Verification

The *Recognize* stage determines whether a dispatched action produced its intended physical effect. No single evidence source suffices in every setting. Visual observations can be occluded, and proprioception cannot fully describe environmental changes. Effect measurements may not distinguish unattempted from failed actions or may require simulator state, task evaluators, or human labels. We therefore organize assessment around semantic, proprioceptive, and effect-level evidence, with embodiment-specific adapters supplying the evidence types available on each platform. When vision-language verification is available, the semantic channel uses a vision-language model to assess the post-execution image $\mathcal{I}_t$ against the intended effect described by $\textsc{Describe}(b_t, k_t, \theta_t)$:

$$\phi_{\text{sem}} = f_{\text{VLM}}(\mathcal{I}_t, \textsc{Describe}(b_t, k_t, \theta_t)) \in \{\text{pass}, \text{fail}, \text{indet}\}. \tag{5}$$

The function Describe converts the action into a verification question, and ϕ_{sem} denotes the resulting judgment. The question may ask whether a mug is held by the gripper or whether the robot reached the goal region. The assessor may return indet when the target is occluded or outside the field of view; this abstention is retained as uncertain evidence. The proprioceptive channel checks embodiment state q against skill-specific state sets $\boldsymbol{Q}$ before and after execution:

$$\phi_{\text{pre}} = \mathbb{1}\left[q_{t-1}^{b_t} \in \boldsymbol{Q}_{b_t,k_t}^{\text{pre}}\right], \qquad \phi_{\text{post}} = \mathbb{1}\left[q_t^{b_t} \in \boldsymbol{Q}_{b_t,k_t}^{\text{post}}\right]. \tag{6}$$

The binary indicators ϕ_{pre} and ϕ_{post} encode state-constraint satisfaction. A violated precondition suggests that the action may no longer apply; a violated postcondition indicates that the required terminal state was not reached. State variables depend on the embodiment: finger joints and palm pose for dexterous hands, gripper width

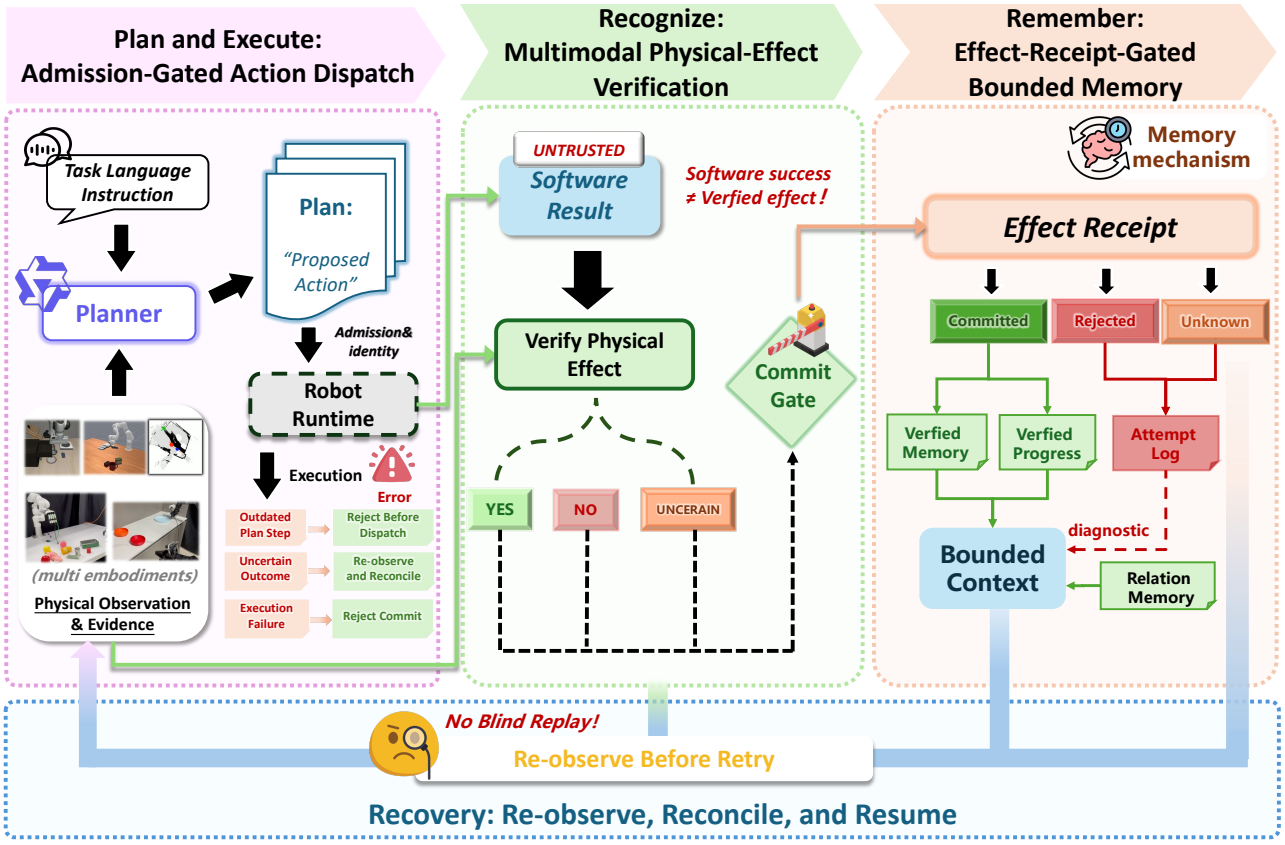


Figure 2: Overview of RoboGuardian for fault-aware execution and memory maintenance across heterogeneous robot embodiments.

and end-effector pose for gripper-based manipulators, and base pose and velocity for mobile robots. The effect-level channel compares the expected target transformation $\Delta\hat{T}_t$ with the measured transformation ΔT_t:

$$\phi_{\text{eff}} = \mathbb{1}\left[d_{\text{p}}(\Delta T_t, \Delta\hat{T}_t) < \epsilon_{\text{p}}^{b_t,k_t} \ \wedge \ d_{\text{r}}(\Delta T_t, \Delta\hat{T}_t) < \epsilon_{\text{r}}^{b_t,k_t}\right]. \quad (7)$$

The functions d_{p} and d_{r} measure translational and rotational discrepancies, with thresholds ϵ determined by the embodiment and skill class. A partial-effect signal $\phi_{\text{eff}}^{\text{part}}$ records motion in the intended direction without goal completion. It informs fusion but cannot update completed progress.

The fusion module collects the runtime status and all available evidence channels in a structured report ρ_t:

$$\rho_t = \left\langle r_t, \phi_{\text{sem}}, \phi_{\text{pre}}, \phi_{\text{post}}, \phi_{\text{eff}}, \phi_{\text{eff}}^{\text{part}} \right\rangle. \quad (8)$$

The fusion function f_{fuse} instantiates the verifier Φ in Eq. (3), producing z_t from the evidence report ρ_t, current observation o_t, and subgoal g_t:

$$z_t = f_{\text{fuse}}(\rho_t, o_t, g_t). \quad (9)$$

Each judgment has a defined transition through the Commit Gate $\mathcal{G}$, which maps it to an Effect Receipt:

$$e_t = \mathcal{G}(z_t) = \begin{cases} \text{committed}, & z_t = \text{yes}, \\ \text{rejected}, & z_t = \text{no}, \\ \text{unknown}, & z_t = \text{uncertain}. \end{cases} \quad (10)$$

Here, e_t denotes the Effect Receipt state. The associated record also stores the action identity, software status, key evidence, and physical judgment. The receipt is the sole interface authorizing trusted memory updates.

3.4 Remember with Receipt-Gated Bounded Memory

The memory mechanism maintains verified memory $\mathcal{M}_t^{\text{ver}}$, verified progress $\mathcal{P}_t^{\text{ver}}$, an attempt log $\mathcal{L}_t$, and relation memory $\mathcal{R}_t$. Verified world facts and task progress are updated only when e_t = committed; rejected or unknown receipts leave trusted state unchanged. The attempt log records the action, evidence, receipt type, and attempt count. A rejected receipt indicates that the intended effect was not achieved, whereas an unknown receipt indicates an unresolved outcome. Relation memory stores object relations, skill relations, and task constraints when available. The system combines these relations with verified facts, verified progress,

and typed diagnostics to form the bounded context:

$$\mathcal{B}_{t+1} = C\big(\mathcal{M}^{\text{ver}}_{t+1}, \mathcal{P}^{\text{ver}}_{t+1}, \mathcal{R}_{t+1}, \mathcal{D}(\mathcal{L}_{t+1})\big)\,. \tag{11}$$

Here, $\mathcal{D}$ extracts diagnostic summaries from $\mathcal{L}_{t+1}$ to inform subsequent planning. These summaries do not constitute verified physical facts. We impose a budget B on $\mathcal{B}_t$. This bounded view preserves recent observations, committed progress, and selected receipt-linked evidence, while complete audit information remains in the attempt log outside the planner context.

3.5 Recovery through Re-observation and Reconciliation

The *Recovery* stage selects the next procedure based on the admission decision, Effect Receipt, attempt log, and bounded context. Every physical retry requires a fresh observation. For an outdated plan step, the system rejects dispatch and regenerates the plan suffix while preserving the committed prefix. For a rejected receipt, it may select an alternative skill, replan, or perform a bounded safe retry. The attempt log prevents repeated use of failed strategies. An unknown receipt alone cannot trigger a new task action. The system first obtains fresh evidence from the adapter, such as a new camera frame, evaluator state, or operator effect label. Given a new observation o'_t, it forms a fresh report ρ'_t and recomputes $z'_t = f_{\text{fuse}}(\rho'_t, o'_t, g_t)$.

A yes result produces a committed receipt and resumes the task; a no result produces a rejected receipt and initiates safe recovery. If uncertainty remains, the system gathers more evidence. After at most N_{obs} re-observation attempts, it marks a still-uncertain outcome as unresolved and requests stronger perception or human confirmation. A non-idempotent skill is never repeated directly from an unknown state, avoiding duplicate grasping, pushing, and placing effects.

3.6 Implementation Details

All embodiments share the same pipeline for planning, verification, receipt gating, memory maintenance, and recovery. Skill libraries, state constraints, effect measurements, and observation tools remain embodiment-specific. In simulation [9, 30, 40], the verifier directly reads object, end-effector, and robot-base poses from the environment. Embodiment states include finger joints, contact states, and palm pose for dexterous hands; gripper width, contact force, and end-effector pose for gripper-based manipulators; and base pose, goal distance, and residual velocity for mobile robots. Model configurations follow each benchmark's native setup. Clutter-Bench uses Qwen3-VL-32B-Instruct [43] with a temperature of 0.2, an 800-token output limit, and 512×512 rendered PNG frames with `low` image detail. We allow up to three provider retries and cap planning at 60 turns. LIBERO uses a frozen SmolVLA [39] checkpoint as its action policy rather than an API-based VLM planner. Navigation experiments retain their native macro-level stack: GPT-4o [22] serves as the language-action model, while Qwen3.5-27B [43] serves as the vision-action model with reasoning output disabled.

For real-world execution, vision and robot sensors jointly estimate physical effects. DINOv2 features [34] provide target point prompts, and SAM [24] extracts object masks. Depth observations are back-projected to estimate rigid transformations between consecutive point clouds. Wider effect tolerances account for perception noise. Within each benchmark, all variants share the same planner or policy, seeds, initial states, and interaction budget. Retry and re-observation limits, memory budgets, effect thresholds, and embodiment constraints are set before evaluation and fixed within each embodiment class.

Table 2: Evaluation metrics and their definitions.

Metric	Definition
Task Success Rate (TSR) ↑	Upstream task-evaluator judgment reported without modification.
Verified Success Rate (VSR) ↑	Rate of task-successful episodes whose execution chain has no outdated plan-step dispatch, duplicate dispatch, or unresolved outcome.
TSR–VSR Gap ↓	Declared successes that the harness cannot support with its own evidence.
Unverified Memory Writes ↓	Outcomes admitted to trusted memory without a verified physical effect.
Duplicate Physical Actions ↓	Logical invocations that produce more than one physical dispatch.
Recovery Coverage ↑	Rate of eligible episodes that enter recovery.
Recovery Success ↑	Rate of recovery cases that are resolved.
Context and Execution Cost ↓	Actions, model calls, wall-clock time, max images per request, and peak request size.

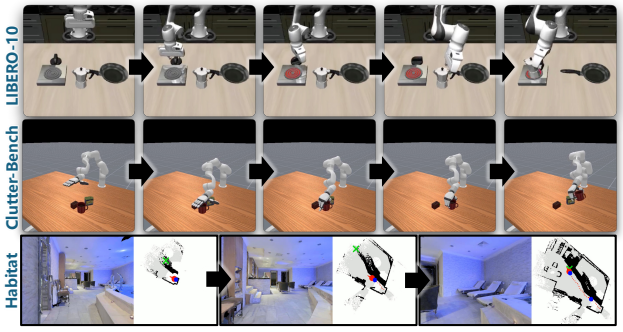


Figure 3: Experimental settings on the three simulated platforms.

4 Experiments

Our experiments address four questions. **Q1:** Does RoboGuardian preserve task capability under fault-free execution? **Q2:** Can it prevent unverified updates and duplicate actions under execution exceptions? **Q3:** Can receipt-gated memory preserve verified progress while bounding context and execution cost? **Q4:** Does the framework transfer across action granularities, task modalities, robot embodiments, and physical platforms?

4.1 Experimental Setup

We evaluate RoboGuardian in three simulated settings and on two physical robot platforms. The simulations cover low-level manipulation on LIBERO-10 [30], skill-level dexterous grasping on Clutter-Bench [9], and vision-language navigation with LaViRA [11] in

Habitat [40]. Clutter-Bench is the ManiSkill-based simulation introduced in AdaClearGrasp [9]. The physical platforms comprise an xArm manipulator with an XHand dexterous hand for tabletop grasping and an ALOHA dual-arm platform with grippers for sequential manipulation. Fig. 3 summarizes the simulated settings. On LIBERO-10, a frozen SmolVLA [39] policy produces low-level actions. This setting addresses Q1 and Q2 through action-level verification of outdated plan steps and uncertain outcomes. Clutter-Bench is the primary quantitative setting for Q1 through Q3, combining skill-level verification, memory ablation, and context analysis. The macro-level LaViRA integration examines cross-modal transfer in Q4. Physical deployments provide qualitative evidence for Q2 and Q4 through perception-derived verification under naturally occurring faults. Together, these settings span low-level actions, high-level skills, navigation episodes, and real-world execution. We state their supported claims separately because evidence sources and integration granularities differ.

Variants. We use three comparison roles, although not all appear on every platform. **Base** denotes the original or ungoverned harness, which writes runtime returns directly into task state and memory. **Ours-Window** is the bounded-window variant used in the Clutter-Bench memory ablation. It supplies only the three most recent images to the planner without receipt-gated memory. **RoboGuardian** maintains receipt-derived task progress and a bounded evidence context, retrieving at most two additional frames linked to the current receipt. Within each platform, we match the planner, skills, policy checkpoint, model, temperature, interaction budget, random seeds, initial states, and fault schedule where applicable. The Clutter-Bench study also includes Original Ada, Window-3, CGEM, and the complete RoboGuardian system.

Metrics. Table 2 summarizes the evaluation metrics. VSR is the primary metric for the action-level and skill-level quantitative studies. Each platform-specific section reports only metrics supported by its integration granularity. Arrows indicate preferred directions.

Fault categories. The experiments cover the three response-oriented exception categories defined in Section 3. An **outdated plan step** applies an action computed from an earlier observation to the current state. An **uncertain outcome** arises when evidence cannot determine whether an action took effect. An **execution failure** is assigned when physical verification returns no. Outdated action chunks in LIBERO instantiate outdated plan steps. Timeout/retry faults in LIBERO and response loss in Clutter-Bench instantiate uncertain outcomes. False software success in Clutter-Bench instantiates execution failure. The clean condition has no injected external faults, although policy or skill failures may still occur. Execution stalls are reported separately as a capability boundary rather than a primary exception category.

4.2 Action-Level Verification on LIBERO-10

We use SmolVLA [39] on LIBERO-10 [30], placing verification between the policy action queue and the environment step. The module checks every low-level action online, and only committed Effect Receipts authorize task-state updates. Base and RoboGuardian use the same reinference frequency and differ only in whether the Commit Gate is enabled. The evaluation spans 10 tasks, 10 initial states, 3 conditions, and 2 variants, totaling 600 episodes. Outdated action chunks and timeout/retry faults are injected at a prevalence of 0.5. Execution stalls are evaluated separately as a secondary condition.

Table 3: LIBERO-10 results with SmolVLA. Each row contains 100 episodes. Actions and wall-clock time are per-episode means and serve as descriptive cost measures.

Variant	Condition	TSR ↑	VSR ↑	Actions ↓	Wall (s) ↓
Base	clean	0.43	0.43	418.6	358.4
RoboGuardian	clean	**0.42**	**0.42**	**395.7**	**349.2**
Base	outdated chunk	0.41	0.20	426.8	367.0
RoboGuardian	outdated chunk	**0.41**	**0.41**	**378.5**	**338.9**
Base	timeout/retry	0.42	0.22	420.4	374.3
RoboGuardian	timeout/retry	**0.39**	**0.39**	**387.8**	**354.9**

Under fault-free execution, Base and RoboGuardian achieve VSRs of 0.43 and 0.42, respectively, indicating negligible capability loss from online verification. This comparable performance separates subsequent fault-handling gains from changes in the underlying policy. RoboGuardian enters recovery in 34 episodes and resolves 7, showing that policy-level failures can occur without injected faults.

Under outdated action chunks, both variants reach TSR 0.41, but Base verifies only 20 successes and produces an unverified write in all 50 exposed episodes. RoboGuardian verifies all 41 successes with no unverified writes, resolves 25 of 67 recovery cases, and reduces the mean action count from 426.8 to 378.5. These results distinguish supported from unsupported successes despite identical upstream TSR.

Under timeout/retry faults, Base duplicates the action in all 50 exposed episodes and verifies only 22 of its 42 successes. RoboGuardian re-observes before retrying, produces no duplicate actions or unverified writes, and verifies all 39 successes. Its slightly lower TSR, 0.39 versus 0.42 for Base, reflects abstention when evidence remains unresolved. This tradeoff avoids classifying uncertain actions as confirmed successes or failures.

Under execution stalls, RoboGuardian enters recovery in 67.5% of eligible episodes and resolves 40.7% of recovery cases. Its VSR of 0.450 is close to Base at 0.425, making this condition a capability boundary rather than a gain. The framework detects stalls and invokes recovery but cannot supply recovery skills absent from the library.

4.3 Skill-Level Verification and Memory Governance

Clutter-Bench [9] provides high-level `push`, `pull`, and `grasp` skills across seven categories, with clutter levels of 2, 4, and 6 distractors. RoboGuardian adds verification and receipt-gated memory at the skill boundary while retaining the environment and skills. False success fabricates a positive return without the intended effect; response loss removes the return after a skill may have executed. We evaluate three conditions over 63 paired scenes each, matching the target, clutter level, scene, random seed, and fault schedule within each pair. The memory ablation uses 30 scenes per condition and is not directly comparable with the main evaluation.

Table 4: Clutter-Bench results over 63 paired scenes per condition. Each pair shares the target, clutter level, scene, random seed, and fault schedule. The pooled row is descriptive.

Condition	Variant	TSR ↑	VSR ↑
Clean	Base	0.857	0.825
Clean	RoboGuardian	**0.905**	**0.857**
False success	Base	0.476	0.460
False success	RoboGuardian	**0.889**	**0.889**
Response loss	Base	0.841	0.825
Response loss	RoboGuardian	**0.889**	**0.873**
Pooled	Base	0.725	0.704
Pooled	RoboGuardian	**0.894**	**0.873**

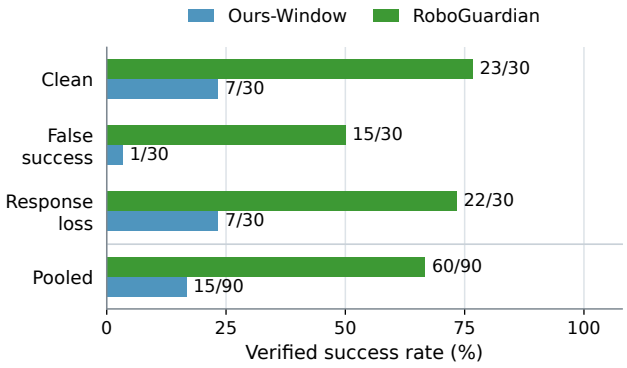


Figure 4: Memory ablation on Clutter-Bench. Bars show VSR. Labels show verified successes among 30 paired scenes per condition and among 90 scenes in the pooled result.

Under fault-free execution, RoboGuardian improves TSR from 0.857 to 0.905 and VSR from 0.825 to 0.857, indicating that skill-level verification preserves normal task capability. Base produces 72 unverified writes across 31 episodes, while RoboGuardian produces none. With no injected faults, these writes arise from naturally occurring planning, perception, or skill errors.

Under false success, Base accepts the fabricated result, causing the planner to mark the grasp subgoal as complete and continue from a false premise. TSR and VSR fall to 0.476 and 0.460. RoboGuardian rejects the result and returns control to the planner for another attempt, reaching 0.889 on both metrics. Verification thus supports recovery as well as outcome certification.

Under response loss, RoboGuardian reaches TSR 0.889 and VSR 0.873, compared with 0.841 and 0.825 for Base. Base produces 32 duplicate actions and 49 unverified writes, while RoboGuardian produces neither by re-observing before issuing another action. Across all conditions, unverified writes fall from 170 across 102 of 189 episodes to zero, showing that trusted memory remains protected under failed and unresolved outcomes.

Figure 4 compares Ours-Window with the receipt-gated CGEM variant. Verified successes increase from 7, 1, and 7 to 23, 15, and 22 across the three conditions. Pooled VSR rises from 15/90 to 60/90, while mean skill dispatches fall from 51.6 to 35.0. Compared with a recency window, receipt-gated memory retains established effects in a compact form and reduces repeated subgoals.

Table 5: Context and execution cost on Clutter-Bench. Actions and model calls are per-episode means. Max images is the largest number of images in any single model request.

Condition	Variant	Actions ↓	Calls ↓	Wall (s) ↓	Max images ↓	Peak request (MB) ↓
Clean	Base	21.0	21.9	654.4	53	21.21
Clean	RoboGuardian	**9.7**	**14.8**	**483.4**	**5**	**2.05**
False success	Base	15.4	16.4	1573.8	42	16.89
False success	RoboGuardian	**9.6**	**12.4**	**696.3**	**5**	**2.04**
Response loss	Base	22.2	23.1	1983.0	53	21.29
Response loss	RoboGuardian	**7.9**	**10.6**	**536.8**	**5**	**2.05**
Pooled	Base	19.5	20.4	1403.7	53	21.29
Pooled	RoboGuardian	**9.1**	**12.6**	**572.2**	**5**	**2.05**

Table 5 shows that RoboGuardian reduces maximum request size from 53 to five images and from about 21.3 MB to 2.05 MB, nearly an order of magnitude. In the pooled results, skill dispatches fall from 19.5 to 9.1, model calls from 20.4 to 12.6, and wall-clock time from 1403.7 s to 572.2 s. Thus, bounded context limits request size while reducing repeated planning and execution.

4.4 Cross-Modal Transfer to VLN

We integrate RoboGuardian with LaViRA [11] on the OpenNav-100 subset of Habitat R2R-CE [25, 40]. Navigation differs from manipulation in horizon and action structure: episodes comprise long sequences of motion decisions, while the native evaluator assesses complete trajectories. We therefore place the governance boundary around each macro-level evaluation rather than individual low-level motion commands. The original navigation stack and evaluator remain unchanged. GPT-4o serves as the language-action model, and Qwen3.5-27B as the vision-action model with reasoning output disabled.

We conduct three independent repeats of 100 episodes each, totaling 300 evaluations, and report native navigation metrics. Success measures whether the agent reaches the goal, while SPL also accounts for path efficiency. nDTW measures agreement between executed and reference trajectories, and SDTW weights this agreement by task success. We omit VSR because the macro-level integration does not produce a receipt chain for individual low-level actions; reporting it would imply an unsupported verification granularity.

Across the three repeats, Success, SPL, nDTW, and SDTW are 0.327 ± 0.040, 0.245 ± 0.032, 0.427 ± 0.015, and 0.226 ± 0.037, respectively. The limited variation indicates stable evaluation records under the integrated audit protocol across runs. The same framework interface operates on navigation episodes despite differences from manipulation in observations, action horizon, and task metrics. These results support macro-level transfer in Q4 but do not demonstrate improved navigation performance or verification of individual motion commands.

4.5 Real-World Validation

The physical deployments assess Q2 and Q4 on two robot embodiments using evidence from real observations rather than simulator state. Fig. 5 summarizes the qualitative results. We examine whether the system rejects false software successes, enforces committed prerequisites for dependent skills, and records observations,

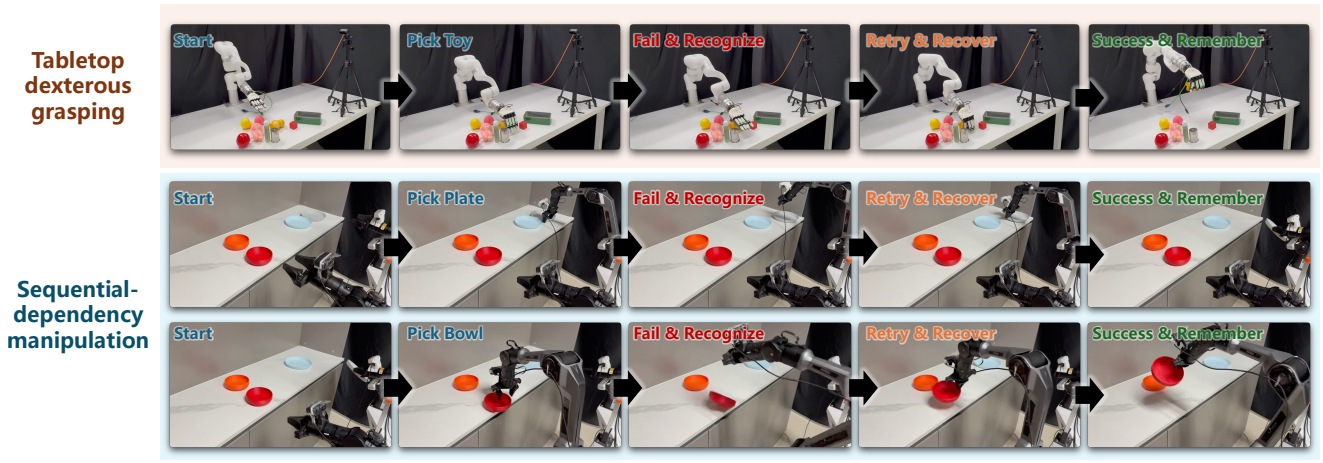


Figure 5: Qualitative experiments on the two physical robot platforms.

judgments, and typed receipts for each run. Videos are provided in the our project website.

Tabletop dexterous grasping. The first deployment executes a tabletop grasping sequence with a dexterous hand. Each skill is verified using camera observations before and after execution. Naturally occurring faults include grasp slippage, occlusion-induced detection errors, and success returns after hand closure without object retention. The video shows a false-success case in which the first `grasp` reports success although the object is not held. BASE accepts the return, writes the unverified effect into trusted state, and proceeds to `lift`. RoboGuardian derives a negative judgment from the before and after evidence, rejects the receipt, and excludes the failed attempt from trusted memory. It then re-observes and commits only after a later grasp is verified. This case distinguishes a software return from a physical fact eligible for commitment.

Sequential-dependency manipulation. The second deployment uses a dual-arm manipulator with grippers, checkpoint-backed high-level skills, and multi-camera observations. A dependent skill is admitted only while the earlier object-retention effect remains committed. This setting tests cases where later slippage invalidates a committed prerequisite. In the first demonstration, the skill for the gray plate reports completion although the robot does not hold the plate. BASE treats the prerequisite as satisfied and dispatches the dependent skill for the red bowl, while RoboGuardian rejects the failed prerequisite and blocks the dependent step. In the second demonstration, the robot initially holds the gray plate and admits the red-bowl skill, but the earlier retention effect is subsequently lost. BASE accepts the final software completion despite the invalid prerequisite. RoboGuardian uses goal-integrity evidence to detect that the gray plate is no longer held and rejects the final result.

5 Conclusion

We present RoboGuardian, a framework for fault-aware execution and memory maintenance in embodied-agent harnesses. It distinguishes outdated plan steps, uncertain outcomes, and execution failures by their required responses. Before dispatch, RoboGuardian checks plan validity and entity identity; after execution, it verifies physical effects using semantic, proprioceptive, and effect-level evidence and maps the resulting judgment to a typed receipt. Only committed receipts update verified memory and task progress; rejected and unknown attempts remain available for diagnosis without altering trusted state. A bounded context organizes verified information and diagnostic summaries for subsequent planning. Unresolved outcomes require re-observation before retrying. Experiments in simulation and on physical robots show little overhead under fault-free execution. Under injected faults, RoboGuardian prevents unverified memory writes and duplicate actions while reducing the per-request context footprint by nearly an order of magnitude on Clutter-Bench.

The reliability of physical-effect verification depends on available evidence, motivating further evaluation under noisy real-world perception. The re-observation limit must also balance safety and latency. The boundary between software reports and trusted physical effects is especially important in shared-state systems, where a single unverified update may mislead all participating agents [21, 41].

Acknowledgments

This work is supported in part by New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0122904), National Natural Science Foundation of China (62192783, 62276128, 62506153, 62606229), Jiangsu Science and Technology Major Project (BG2025035), the Fundamental Research Funds for the Central Universities(KG202514), Postgraduate Research & Practice Innovation Program of Jiangsu Province (26CXJH0340) and the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691* (2022).

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 2024. π_0:

A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164* (2024).

[3] Irvin Steve Cardenas, Marcus Anthony Arnett, Natalie Catherine Yeo, Lucky Sah, and Jong-Hoon Kim. 2026. ROSClaw: An OpenClaw ROS 2 framework for agentic robot control and interaction. *arXiv preprint arXiv:2603.26997* (2026).

[4] Yangtao Chen, Zixuan Chen, Nga Teng Chan, Junting Chen, Junhui Yin, Jieqi Shi, Yang Gao, Yong-Lu Li, and Jing Huo. 2025. Robohiman: A hierarchical evaluation paradigm for compositional generalization in long-horizon manipulation. *arXiv preprint arXiv:2510.13149* (2025).

[5] Zixuan Chen, Xiangrong Feng, Jieqi Shi, Lin Shao, Jing Huo, and Yang Gao. 2026. AGiLe: Learning Robust Long-Horizon Manipulation via Affordance-Grounded Bidirectional Latent Planning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6760–6769.

[6] Zixuan Chen, Chongkai Gao, Lin Shao, Jieqi Shi, Jing Huo, and Yang Gao. 2026. ManiLong-Shot: Interaction-aware one-shot imitation learning for long-horizon manipulation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 18189–18197.

[7] Zixuan Chen, Jing Huo, Yangtao Chen, and Yang Gao. 2025. Robohorizon: An llm-assisted multi-view world model for long-horizon robotic manipulation. *arXiv preprint arXiv:2501.06605* (2025).

[8] Zixuan Chen, Junhui Yin, Yangtao Chen, Jing Huo, Pinzhuo Tian, Jieqi Shi, Yiwen Hou, Yinchuan Li, and Yang Gao. 2026. Deco: Task decomposition and skill composition for zero-shot generalization in long-horizon 3d manipulation. *IEEE Robotics and Automation Letters* (2026).

[9] Zixuan Chen, Wenquan Zhang, Jing Fang, Ruiming Zeng, Zhixuan Xu, Yiwen Hou, Xinke Wang, Jieqi Shi, Jing Huo, and Yang Gao. 2026. AdaClearGrasp: Learning Adaptive Clearing for Zero-Shot Robust Dexterous Grasping in Densely Cluttered Environments. *arXiv preprint arXiv:2603.10616* (2026).

[10] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. 2025. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research* 44, 10-11 (2025), 1684–1704.

[11] Hongyu Ding, Ziming Xu, Yudong Fang, You Wu, Zixuan Chen, Jieqi Shi, Jing Huo, Yifan Zhang, and Yang Gao. 2025. LaViRA: Language-Vision-Robot Actions Translation for Zero-Shot Vision Language Navigation in Continuous Environments. *arXiv preprint arXiv:2510.19655* (2025).

[12] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. 2023. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378* (2023).

[13] Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando De Freitas, and Serkan Cabi. 2023. Vision-language models as success detectors. *arXiv preprint arXiv:2303.07280* (2023).

[14] Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. 2024. Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371* (2024).

[15] Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. 2023. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. *arXiv preprint arXiv:2307.00329* (2023).

[16] Songjia He, Zixuan Chen, Hongyu Ding, Dian Shao, Jieqi Shi, Chenxu Li, Jing Huo, and Yang Gao. 2026. V-Dreamer: Automating Robotic Simulation and Trajectory Synthesis via Video Generation Priors. *arXiv preprint arXiv:2603.18811* (2026).

[17] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*. PMLR, 9118–9147.

[18] Wenlong Huang, Fei Xia, Dhruv Shah, Danny Driess, Andy Zeng, Yao Lu, Pete Florence, Igor Mordatch, Sergey Levine, Karol Hausman, et al. 2023. Grounded decoding: Guiding text generation with grounded models for embodied agents. *Advances in Neural Information Processing Systems* 36 (2023), 59636–59661.

[19] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. 2022. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608* (2022).

[20] Xu Huang, Weixin Mao, Yinhao Li, Hua Chen, and Jiabao Zhao. 2026. Long-Term Memory for VLA-based Agents in Open-World Task Execution. *arXiv preprint arXiv:2604.15671* (2026).

[21] Dongjie Huo, Haoyun Liu, Guoqing Liu, Dekang Qi, Zhiming Sun, Maoguo Gao, Jianxin He, Yandan Yang, Xinyuan Chang, Feng Xiong, et al. 2026. ABot-Claw: A foundation for persistent, cooperative, and self-evolving robotic agents. *arXiv preprint arXiv:2604.10096* (2026).

[22] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).

[23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246* (2024).

[24] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C Berg, Wan-Yen Lo, et al. 2023. Segment anything. In *Proceedings of the IEEE/CVF international conference on computer vision*. 4015–4026.

[25] Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. 2020. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*. Springer, 104–120.

[26] Chenxu Li, Zixuan Chen, Yetao Li, Jiapeng Xu, Hongyu Ding, Jieqi Shi, Jing Huo, and Yang Gao. 2026. MoMaStage: Skill-State Graph Guided Planning and Closed-Loop Execution for Long-Horizon Indoor Mobile Manipulation. In *RSS 2026 Workshop on Human-centric Mobile Manipulation*.

[27] Ruiying Li, Yunlang Zhou, YuYao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, et al. 2026. Roboclaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558* (2026).

[28] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. Code as policies: Language model programs for embodied control. In *2023 IEEE International conference on robotics and automation (ICRA)*. IEEE, 9493–9500.

[29] Kevin Lin, Christopher Agia, Toki Migimatsu, Marco Pavone, and Jeannette Bohg. 2023. Text2motion: From natural language instructions to feasible plans. *Autonomous Robots* 47, 8 (2023), 1345–1365.

[30] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems* 36 (2023), 44776–44791.

[31] Haowen Liu, Xirui Li, Shaoxiong Yao, Peng Shi, Tianyi Zhou, Jia-Bin Huang, Furong Huang, and Jiayuan Mao. 2026. Guava: An Effective and Universal Harness for Embodied Manipulation. *arXiv preprint arXiv:2606.18363* (2026).

[32] Songyang Liu, Chaozhuo Li, Chenxu Wang, Jinyu Hou, Zejian Chen, Litian Zhang, Zheng Liu, Qiwei Ye, Yiming Hei, Xi Zhang, et al. 2026. Clawkeeper: Comprehensive safety protection for openclaw agents through skills, plugins, and watchers. *arXiv preprint arXiv:2603.24414* (2026).

[33] Zeyi Liu, Arpit Bahety, and Shuran Song. 2023. Reflect: Summarizing robot experiences for failure explanation and correction. *arXiv preprint arXiv:2306.15724* (2023).

[34] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. 2024. Dinov2: Learning robust visual features without supervision. *Transactions on Machine Learning Research Journal* (2024).

[35] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[36] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*. 1–22.

[37] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. 2023. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. *arXiv preprint arXiv:2307.06135* (2023).

[38] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems* 36 (2023), 8634–8652.

[39] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, et al. 2025. Smolvla: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844* (2025).

[40] Andrew Szot, Alexander Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Singh Chaplot, Oleksandr Maksymets, et al. 2021. Habitat 2.0: Training home assistants to rearrange their habitat. *Advances in neural information processing systems* 34 (2021), 251–266.

[41] Huajie Tan, Xiaoshuai Hao, Cheng Chi, Minglan Lin, Yaoxu Lyu, Mingyu Cao, Dong Liang, Zhuo Chen, Mengsi Lyu, Cheng Peng, et al. 2025. Roboos: A hierarchical embodied framework for cross-embodiment and multi-agent collaboration. *arXiv preprint arXiv:2505.03673* (2025).

[42] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291* (2023).

[43] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[44] Shunyu Yao, Jeffrey Zhao, Dian Yu, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In *NeurIPS 2022 Foundation Models for Decision Making Workshop*.

[45] Takuma Yoneda, Jiading Fang, Peng Li, Huanyu Zhang, Tianchong Jiang, Shengjie Lin, Ben Picker, David Yunis, Hongyuan Mei, and Matthew R Walter. 2024. Statler: State-maintaining language models for embodied reasoning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 15083–15091.

[46] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint*

arXiv:2304.13705 (2023).

[47] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 19724–19731.

[48] Enshen Zhou, Qi Su, Cheng Chi, Zhizheng Zhang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and He Wang. 2025. Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 6919–6929.

[49] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*. PMLR, 2165–2183.